ScanFlow Mosaic

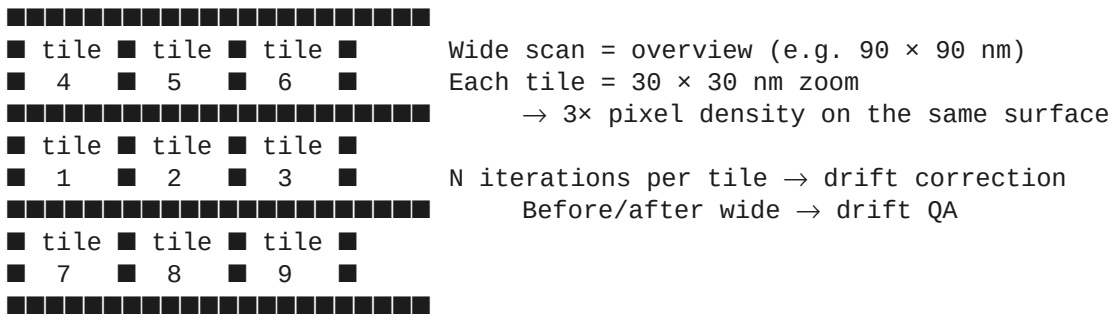
From “build a mosaic routine” to a working pipeline

What I shipped this week, and the Createc COM rabbit hole that ate four days of it.

SPMQT-Lab — Week of 2026-05-18

What is a mosaic, and why?

Wide + 9 tiles + verification, all automated.



- Atomic detail across a 90 nm field in one automated run
- Manually re-framing 9 times = no thanks
- Each tile scanned 3 times → drift-correct in-place via cross-correlation
- Wide-before / wide-after = QA of total drift during the run

---

SPEAKER NOTES

Mosaic = overview + tiles + verification. The value: I get atomic detail across a 90 nm field in a single automated run instead of manually re-framing 9 times. Before/after wide images tell me how much the sample drifted during the whole campaign. Each tile is scanned 3 times so the cross-correlation between iterations can drift-correct in place.

Round 1 — three failures in a row

Three days in, nothing working.

- ■ Tile 1 landed *outside* the wide field by ~100 nm.
- ■ X dimension kept resetting to 2 nm on every scan.
- ■ STMAFM crashed after force-stopping a run.
- ■ getxypos() returned None on this STMAFM build.

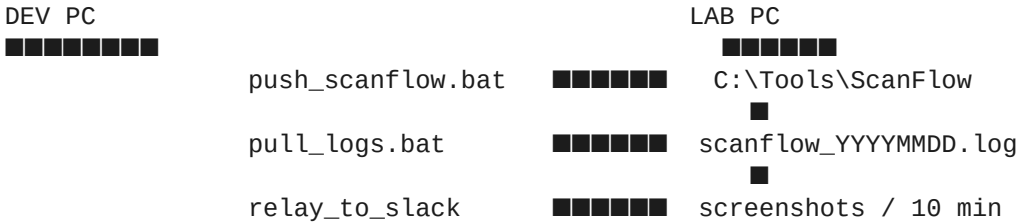
Common pattern: documented Createc primitives behave unpredictably on this firmware. No way to debug without instrumentation.

SPEAKER NOTES

First implementation used set_offset_image_coord for tile positioning — the documented Createc primitive. Tiles landed outside the wide field by 100+ nm. Second attempt: nudge_offset_pixels. Same kind of failure, different magnitude. Meanwhile every scan kept setting X = 2 nm because the tuple-form setp('SCAN.IMAGESIZE.NM', (x, y)) corrupts the X side on this rig. Stop hung. Crashes after Force Quit. And I couldn't even read the current XY position because getxypos() requires a secondary COM dispatch that doesn't exist on our STMAFM build. By Monday evening I had nothing working.

Building the diagnostic toolchain

Stop guessing. Start measuring.



- **push_scanflow.bat** — robocopy syncs code edits dev → lab over LAN
- **pull_logs.bat** — robocopy pulls daily 5 MB rolling logs lab → dev
- **relay_to_slack.py** — screenshots every 10 min posted to a channel
- **Auto file logger** inside ScanFlow — every COM call's outcome

---

SPEAKER NOTES

I couldn't debug what I couldn't see. So I built three pieces of infrastructure. push_scanflow.bat — robocopy syncs my code edits over the LAN. pull_logs.bat — pulls rolling daily log files back to me. relay_to_slack.py — screenshots every 10 minutes during long runs, posted to a Slack channel so I can check from anywhere. Plus an automatic file logger inside ScanFlow that records every COM call's outcome — about 5 MB per day. With these in place I stopped guessing what the rig was doing and started reading what it actually did.

Positioning Test tab — the empirical probe

One COM call at a time. Read the result. Repeat.

```
=== Probing XY-read mechanisms ===
  getp('SCAN.OFFSET.X.NM',   'SCAN.OFFSET.Y.NM')   → X='1.183', Y='25.094'   ✓
  getp('SCAN.OFFSET.X.VOLT', 'SCAN.OFFSET.Y.VOLT') → X='0.12',  Y='2.61'    ✓
  getp('STMAFM.XYOFF.X.NM',  'STMAFM.XYOFF.Y.NM')  → X=(empty), Y=(empty)
  getp('SCAN.XYOFF.X.VOLT',  'SCAN.XYOFF.Y.VOLT')  → X=(empty), Y=(empty)
  getdacvalf(board=0, ch=0)                        → ERROR: no such attribute
  ...
```

- Probed 12 candidate getp keys + 8 DAC channels
- **SCAN.OFFSET.{X,Y}.NM** mirrors STMAFM's display (in nm)
- **SCAN.OFFSET.{X,Y}.VOLT** reports the piezo voltage
- **setxyoffvolt** takes piezo *volts* despite the name — ~10 nm/V on our rig

---

SPEAKER NOTES

I built a diagnostic panel that fires one COM call at a time — pick a command, type a value, click Send, read the result. Probed 12 candidate getp keys and 8 DAC channels. Found that SCAN.OFFSET.X.NM and SCAN.OFFSET.Y.NM correctly mirror what STMAFM displays, in nanometres. Also found that setxyoffvolt — despite the name — takes its arguments in volts, not nm, with a piezo gain of about 10 nm per volt on our rig. That was the unlock: a verified read primitive and a verified write primitive, with a known unit relationship between them.

The calibration story

Read in nm, write in volts. We need V/nm.

```
set_offset_nm(x_nm, y_nm)
    = setxyoffvolt(x_nm × V/nm_x,
                   y_nm × V/nm_y)
```

- **From-current:** divide .VOLT by .NM. Works when offset is large.
- **Failure mode:** Createc rounds .VOLT to 2 dp → near-origin reading is unreliable.
- **Symptom:** Y = 0.04 V / 0.414 nm → 7% off → 2 nm tile mis-placement.
- **Fix:** delta-probe — send a known +0.1 V move, read the .NM delta, derive V/nm.
- **Result:** <1% accuracy from any starting position, including XY = (0, 0).

SPEAKER NOTES

Read in nm, write in volts. I need V/nm. First attempt: derive from the current offset — divide the .VOLT reading by the .NM reading. Worked when both axes had a healthy starting offset. But Createc rounds .VOLT to two decimal places — so when Y started at 0.4 nm with .VOLT showing 0.04, the ratio was 7% off, and tile Y landed 2 nm off target. The fix: a delta-probe. ScanFlow sends a known +0.1 V move (~1 nm displacement), measures the resulting .NM delta, derives V/nm from the change, and restores the original position. The signal is 20× larger than the rounding error, so calibration is accurate to better than 1% no matter where you start — including exactly XY = (0, 0).

Breakthrough — tile 01 lands on target

Same code, calibrated.

Before:

```
tile 01: set_offset_nm(-19.052, -20.765)
tile 01 after positioning: offset X=-182.899, Y=-199.344    ← 10× off!
```

After:

```
XY calibration: 0.10436 V/nm (X), 0.10436 V/nm (Y)
tile 01: target  X=-21.184   Y=-29.586
tile 01: actual  X=-21.184   Y=-29.586   (err: +0.000, +0.000)  ✓
```

Sub-Ångström accuracy on every tile, every run. The breakthrough wasn't a clever algorithm — it was understanding what the COM actually does and accepting that what's labelled *volts* really is volts.

SPEAKER NOTES

Before calibration: requested -19 nm, got -183 nm. After: requested -21 nm, got -21 nm. Sub-Ångström accuracy on every tile, every run. The breakthrough wasn't a clever algorithm — it was understanding what the COM actually does and accepting that what's labelled 'volts' really is volts.

Drift correction in the working mosaic

Two layers: per-tile + per-campaign.

```
per tile:
    iter 1   → save as the tile's reference
    iter 2   → cross-correlate vs iter 1 → set_offset_nm(current + drift)
    iter 3   → cross-correlate vs iter 1 → set_offset_nm(current + drift)

end of mosaic:
    wide_after vs wide_before → measure total sample drift
```

- Per tile: cross-correlate iter N vs iter 1; convert px-shift → nm-shift via tile nm/px
- Re-centre via set_offset_nm — calibrated, verified write
- End of campaign: wide_before vs wide_after → total sample drift over the whole run
- Useful as an overnight rig stability metric

SPEAKER NOTES

Drift correction has two layers in the working mosaic. Inside each tile, iterations 2 and 3 cross-correlate against iter 1, derive the pixel shift, convert to nm using the calibrated tile-frame nm-per-pixel, and use set_offset_nm to re-centre the next scan. At the end of the campaign, ScanFlow cross-correlates wide_before vs wide_after to report total sample drift over the entire run — useful as an overnight stability metric.

Tile grid layout — middle row first

Tile 1 shares Y with wide; tile 2 is the wide centre itself.

```
           col -1    col 0    col +1

row -1       4         5         6        ← top row, scanned 4th–6th
row  0       1         2         3        ← middle row (same Y as wide), 1st
row +1       7         8         9        ← bottom row, scanned 7th–9th
```

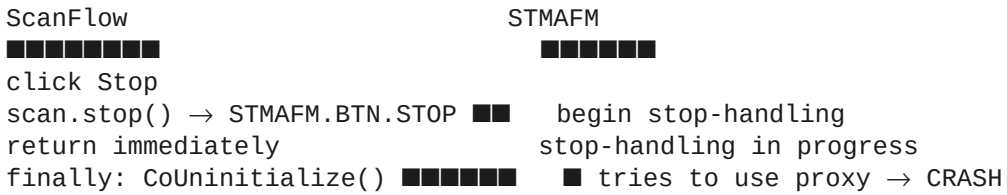
- **Old:** tile 1 = top-left corner (both X and Y shifted)
- **New:** tile 1 = middle-left (only X shifted)
- Middle horizontal strip scanned first — least accumulated drift
- Within-row order: left → right unchanged

SPEAKER NOTES

Tile 1 used to be top-left corner. I changed the row order so the middle row goes first — tiles 1–3 share the wide image's Y axis, and tile 2 is the wide centre itself. The reasoning: the middle horizontal strip is usually what you care about most, and by scanning it first we capture it with the least accumulated drift. Top and bottom rows come after.

STMAFM crash after Stop — fixed

Classic Win32 COM race condition.



- Wait for SCANSTATUS to confirm idle before returning from the wait loop
- 250 ms sleep before CoUninitialize() drains in-flight COM events
- force_stop() waits 8 s for graceful exit before resorting to QThread.terminate()
- Soft Stop now leaves STMAFM in a clean state

SPEAKER NOTES

STMAFM sometimes crashed after I stopped a run. Classic Win32 COM race: we send STOP, return immediately, the worker thread tears down its COM apartment while STMAFM is still mid-stop, holding our proxy. Three fixes. One: after sending STMAFM.BTN.STOP, poll SCANSTATUS until it confirms idle, then return. Two: a 250 ms sleep before CoUninitialize to drain in-flight events. Three: force_stop waits 8 seconds for graceful exit before resorting to QThread.terminate — the only one that's still unsafe, with a warning that STMAFM may need a restart afterwards.

Other things shipped this week

Beyond the mosaic itself.

- **Z Stability tab** — live Z-piezo drift plot, rolling-window stats (5 min, 1 h, 3 h)
- **Auto file logging** — daily rolling 5 MB log file, no user action
- **Sweep voltage warnings** — confirmation dialog for |V| > 1 V or < 100 mV
- **Settling delays** — configurable pause between scans for piezo creep
- **Stop / Emergency / Force Quit** — three-tier halt with log lines at each level
- **Editable step preview** — table view of full sweep before launching
- **Slack screenshot relay** — every 10 min during long runs
- **Pull-logs script** — robocopy lab logs back to dev WSL for analysis

---

SPEAKER NOTES

Beyond the mosaic itself, a handful of supporting features dropped this week. Z stability tab plots drift in real time with rolling-window statistics. Auto file logging means every COM call's outcome is captured for later analysis. Voltage warnings — high (greater than 1 V) and low (less than 100 mV) — require explicit confirmation now. Settling delays let the piezo creep settle between scans. Three-tier stop with logged intent at every level. Editable sweep preview so I can tweak per-step parameters before launching an overnight run. And screenshot relay to Slack so I can monitor a 4-hour run from anywhere.

Lessons, current state, next steps

End-of-week debrief.

What I learned	Where we are	What's next
Createc COM is poorly documented; empirical probing is essential.	Mosaic ships and works: wide + 9 × 3 tiles + wide, with per-tile drift correction.	Between-tile drift correction so 9 tiles stitch into one sharp picture
Two-decimal readback rounding bites you at small values.	Calibration is automatic and accurate to <1% from any starting position.	ProbeFlow integration for polished post-processing → PPTX export
Diagnostic infrastructure pays for itself in one session.	XY positioning works for any future routine that needs it.	UniMR-style classification on detected molecules.
QThread.terminate + COM = always wrong.	STMAFM no longer crashes on Stop.	Configurable grid sizes (1 × N strip, 5 × 5, …).

Three lessons: nothing replaces empirical probing on undocumented hardware; small precision issues compound; Win32 COM cleanup matters.

SPEAKER NOTES

Three lessons. One: nothing replaces empirical probing on undocumented hardware — I spent a day guessing what COM calls did and another fifteen minutes finding out for real with a diagnostic panel. Two: small precision issues compound — Createc rounding .VOLT to two decimal places caused a 7% miscalibration that caused a 2 nm tile mis-placement. Three: Win32 COM cleanup matters; QThread.terminate is the wrong tool for any COM-using thread. Current state: mosaic works, calibration is automatic, crashes are gone. Next: between-tile drift correction so the 9 tiles stitch into a single sharp image, ProbeFlow integration for PPTX export, and adapting the XY positioning primitives into Survey for proper molecule-by-molecule auto-zoom. Thank you — questions?